

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

CE 4301 — Arquitectura de Computadores I

Especificación de Arquitectura ISA

Profesor: Dr.-Ing. Jeferson González Gómez

Estudiantes:

Meibel Elena Mora Brenes 2021128644

Yendry Badilla Gutiérrez 2016113047

Sergio Marín

Braulio Retana

Fecha de entrega: 4 de mayo de 2026

Entrega: TecDigital, antes de las 11:59 pm

Índice

1. Introducción	3
1.1 Objetivos de diseño	3
2. Arquitectura General	4
2.1 Organización de la memoria	4
2.2 Tipos de memoria	4
2.3 Datapath General	4
3. Convención de Registros	6
3.1 Tabla de Registros	6
3.2 Registros de Estado	6
3.2.1 Estructura del registro	6
3.2.2 Actualización de las flags	7
4. Formato de Instrucción	7
4.1 Formato R (Register-to-Register)	7
4.2 Formato I (Instrucciones con inmediato)	8
4.3 Formato M (Memoria)	8
4.4 Formato J (Jump/Branch)	8
4.5 Formato K (Bóveda de Llaves)	9
4.6 Formato T (TEA)	9
5. Set de Instrucciones	9
5.1 Instrucciones Aritméticas	9
5.2 Instrucciones Lógicas y desplazamiento	10
5.3 Instrucciones de Memoria	10
5.4 Instrucciones de Control de flujo	10
5.5 Instrucciones Especializadas (Seguridad)	11
5.6 Instrucciones Especializadas (TEA)	11
5.7 Instrucción nula	12
6. Detalles de ejecución y Comportamiento	12
6.1 Flujo general del sistema	12
6.1.1 Carga de archivos en memoria	12
6.1.2 Ejecución del CPU	13
6.1.3 Autenticación y seguridad	13
6.1.4 Proceso de cifrado	13

6.1.5 Extracción de resultados	13
7. Direccionamiento	14
7.1 Modos de Direccionamiento	14
8. Seguridad y Extensiones Criptográficas	14
8.1 Bit de Autenticación (AUTH)	14
8.2 Bóveda de Llaves (Key Vault)	14
8.2.1 Organización de la bóveda	14
8.2.2 Registros seguros internos de la Bóveda	14
8.2.3 Instrucciones Asociadas	15
8.3 Mecanismo de Autenticación.....	15
8.3.1 Funcionamiento del módulo	15
8.3.2 Parámetros de Seguridad	15
9. Conclusiones	16

1. Introducción

El ISA RISC TEA Vault es un Instruction Set Architecture (ISA) personalizado de 32 bits basado en arquitectura RISC. Este ISA incorpora extensiones de hardware especializadas para seguridad informática, incluyendo:

- Instrucciones nativas para el algoritmo de cifrado TEA (Tiny Encryption Algorithm)
- Acceso a una bóveda de Llaves Criptográficas (Key Vault) segura.
- Arquitectura de 16 registros de propósito general de 32 bits cada uno.
- Memoria direccionable de 32 bits con capacidad mínima de 64KB.
- Arquitectura Harvard: Memorias separadas para instrucciones y datos, permitiendo acceso paralelo.

1.1 Objetivos de diseño

1. Proporcionar un ISA personalizado, simple y educativo basado en principios RISC.
2. Integrar extensiones de seguridad sin comprometer la simplicidad.
3. Permitir eficiencia en operaciones criptográficas mediante instrucciones especializadas.
4. Mantener compatibilidad con herramientas de simulación estándar (Icarus Verilog).

2. Arquitectura General

2.1 Organización de la memoria

El procesador implementa una arquitectura Harvard simplificada con dos espacios de memoria independientes:

Esta separación permite:

- Acceso simultáneo a instrucción y datos
- Pipeline más eficiente sin conflictos de acceso a memoria
- Evitar conflictos entre fetch y acceso de datos
- Simplificar el diseño del CPU

2.2 Tipos de memoria

1. Memoria de instrucciones

- Memoria de palabras de 32 bits
- Tiene tamaño parametrizable (INSTR_MEM_SIZE).
- Se inicializa a través del archivo “program.mem”.
- Cada entrada contiene una instrucción completa.
- El acceso se realiza mediante el Program Counter (PC).

PC	Indice	Instrucción
0	0	instr_mem[0]
4	1	instr_mem[1]
8	2	instr_mem[2]

2. Memoria de datos (RAM)

- Es una memoria byte-addressable.
- Tiene un tamaño fijo de 65536 bytes = 64 KB .
- El acceso a datos se hace mediante las operaciones de carga (LD) y almacenamiento (ST) trabajan con palabras de 32 bits.

Esto implica que el acceso se hace en bloques de 4 bytes, alineado a palabras de 32 bits. Además, la memoria se inicializa mediante el archivo “data.mem”.

2.3 Datapath General

El módulo datapath implementa la ruta principal de datos del procesador y es responsable de ejecutar las operaciones definidas por la ISA. Este módulo integra el banco de registros, la unidad aritmético-lógica (ALU), la lógica de selección de operandos, el mecanismo de escritura de resultados(write-back) y el registro de estado.

Estructura del Datapath

El datapath está compuesto por los siguientes bloques:

- Register File(register_file): almacena los registros generales del procesador. Permite leer hasta tres operandos (rs1, rs2, rs3) y escribir un resultado en el registro destino (rd).

- ALU (alu): ejecuta operaciones aritméticas, lógicas, desplazamientos y operaciones especiales como las utilizadas en el cifrado.
- Multiplexor de operandos (ALU input B): selecciona entre un valor de registro(rs2_data) o un valor inmediato.
- Multiplexor de write-back: selecciona el dato que será escrito en el banco de registros.
- Registro de estado(status_reg): almacenas banderas del procesador como Zero, Carry, Overflow, Auth y Exception.

Flujo de datos del datapath

El flujo de ejecución dentro del datapath es el siguiente:

1. Se leen los operandos desde el banco de registros (rs1, rs2, rs3).
2. Se selecciona el segundo operando de la ALU (registro o inmediato).
3. La ALU ejecuta la operación indicada por señal **alu_op**.
4. El resultado se envía al multiplexor de write-back.
5. El valor seleccionado se escribe en el registro destino (rd).
6. Las banderas generadas por la ALU se almacenan en el registro de estado.

Selección de operandos en el datapath

El datapath permite ejecutar operaciones tanto con registros como con valores inmediatos mediante la señal de control alu_src_b:

Esto permite soportar instrucciones como:

- Operaciones entre registros (ADD, SUB, AND, etc).
- Operaciones con inmediato (SRLLI, SLLI, MOV inmediato)

Mecanismo de Write-Back

El resultado se escribe en el banco de registros se selecciona mediante la señal mem_to_reg, donde si es cero es un resultado proveniente de la ALU, y si es 1, es un dato proveniente de memoria (LD). Esto permite integrar correctamente las instrucciones de carga (LD).

Integración con seguridad

El datapath recibe el estado de autenticación desde el módulo **auth_unit** mediante la señal **auth_status_in**. Este valor es utilizado por ALU para validar la ejecución de instrucciones privilegiadas. Además, el datapath combina errores provenientes de la ALU y de la bóveda de llaves.

Esto permite:

- Operaciones inválidas.
- Accesos no autorizados a la bóveda.

Banderas de Estado

La ALU genera un conjunto de banderas(alu_flags), entre las cuales destaca la bandera Zero, utilizada para instrucciones de salto condicional como BEQ.

Estas banderas se almacenan en el registro de estado, junto con:

- Bit de autenticación (AUTH)
- Bit de excepción (EXC)

3. Convención de Registros

El procesador cuenta con 16 registros de propósito general, cada uno de 32bits. Están direccionados por un índice de 5 bits (0–15).

El acceso a los registros se realiza mediante tres puertos de lectura (rs1, rs2, rs3) y un puerto de escritura(rd), lo que permite ejecutar operaciones con múltiples operandos en una sola instrucción.

Tipos de Acceso a los registros:

Lectura: Las lecturas son asíncronas, lo que significa que el valor del registro se obtiene inmediatamente al cambiar la dirección.

Escritura: La escritura es sincrónica, y ocurre únicamente en el flanco positivo del reloj cuando la señal we (write enable), está activa.

Reset: En caso de reset, todos los registros se inicializan en cero.

3.1 Tabla de Registros

Índice	Nombre	Ancho	Propósito
r0	zero	32 bits	Constante de valor cero por convención.
r1-r15	gen	32 bits	Registros de propósito general.

A diferencia de arquitecturas como RISC-V, en este diseño no se asignan roles fijos a los registros. Todos los registros son de propósito general, permitiendo flexibilidad en la ejecución de instrucciones.

3.2 Registros de Estado

Almacena información relevante sobre el resultado de las operaciones del procesador, así como el estado de seguridad del sistema. Este registro permite que la unidad de control tome decisiones basadas en condiciones aritméticas y eventos de seguridad.

Estructura del registro:

3.2.1 Estructura del registro

El registro tiene 6 bits, organizados de la siguiente forma:

[5] Exc | [4] AUTH | [3] V | [2] C | [1] N | [0] Z

Bit	Flag	Descripción
0	Z (Zero)	Indica si el resultado de la operación es zero.

1	N (Negative)	Indica si el resultado es negativo (bit más significativo en 1).
2	C (Carry)	Indica acarreo en operaciones aritméticas.
3	V (Overflow)	Indica desbordamiento aritmético en operaciones signed.
4	AUTH (AUTHENTICATION)	Indica si el procesador está autenticado (1) o no (0).
5	EXC (Exception)	Indica que ocurrió una exception (operación ilegal o acceso no autorizado a la bóveda).

3.2.2 Actualización de las flags

Flags de la ALU

Las banderas aritméticas (Z, N, C, V) se actualizan automáticamente después de cada operación de la ALU cuando la señal “we” está activa.

Flags de autenticación

El flag AUTH se actualiza con base en la salida del módulo de autenticación (auth_unit), indicando si el procesador tiene permisos para ejecutar instrucciones privilegiadas.

Flags de Exception

El flag de EXC se activa cuando ocurre una operación ilegal o un intento no autorizado a la bóveda de llaves.

El registro de estado integra directamente el estado de seguridad del procesador mediante los flags AUTH y EXC, permitiendo que las instrucciones privilegiadas sean controladas a nivel de hardware. Esto garantiza que operaciones sensibles, como el acceso a la bóveda de llaves o el uso del módulo criptográfico, solo se ejecuten bajo condiciones autorizadas.

4. Formato de Instrucción

Todas las instrucciones de la ISA tienen un tamaño fijo de 32 bits. El campo opcode ocupa los bits [31:27], por lo que la ISA permite hasta 32 instrucciones distintas. Aunque el register_file utiliza direcciones de 5 bits para soportar 32 registros, la codificación actual usa campos de 4 bits dentro de la instrucción y los extiende con ceros en el decoder para compatibilidad con el banco de registros.

4.1 Formato R (Register-to-Register)

Usado para operaciones entre tres registros.

Campos:

- OpCode [31:27]: Código de operación (5 bits) — especifica la operación

- RD [26:23]: Primer Operando Fuente (3 bits)
- RS1 [22:19]: Segundo operando fuente (3 bits)
- RS2 [18:15]: Tercer operando fuente (3 bits)
- [14:0]: Sin uso

Aplica a las instrucciones: ADD, SUB, OR, XOR, SRL, SLL, MUL, CMP, AND, NOP

Nota: Se agrega la instrucción NOP, que modifica todos los campos y quedan en 0 que son los valores por defecto.

Ejemplo: ADD, r1, r2, r3

4.2 Formato I (Instrucciones con inmediato)

Usado por instrucciones que requieren un valor inmediato.

Campos:

- OpCode [31:27]: Código de operación (5 bits)
- RD [26:23]: Registro operando fuente (3 bits)
- RS1 [22:19]: Registro operando fuente (3 bits)
- imm [18:0]: Valor inmediato de 19 bits

Aplica a: MOV, SRLI, SLLI

Ejemplo: MOV r0, #0XA5A5

4.3 Formato M (Memoria)

Usado para acceso a memoria RAM mediante LD y ST.

- OpCode [31:27]: Código de operación (5 bits)
- RD [26:23]: Registro operando fuente (3 bits)
- RS2 [26:23]: Registro operando fuente (3 bits)
- imm [18:0]: Valor inmediato de 19 bits.

En LD, el campo rd indica el registro destino.

En ST, el campo rs2 se interpreta como el registro fuente cuyo valor se guarda en memoria.

4.4 Formato J (Jump/Branch)

Usado para instrucciones de salto y rama.

Campos:

- OpCode [31:27]: Código de operación (5 bits)
- Rs1[26:23]: Registro operando fuente
- Rs2[22:19]: Registro operando fuente
- Imm[18:0]: Valor inmediato de 19 bits

Aplica para: BEQ y JMP

Ejemplo: BEQ r1, r2, etiqueta

4.5 Formato K (Bóveda de Llaves)

Usado por instrucciones de autenticación y manejo de la bóveda.

Campos:

- OpCode [31:27]: Código de operación (5 bits)
- Slot/kdest[26:24]: cual llave, de 0 a 3
- palabra [23:21]: que palabra (32 bits) de la llave de 0 a 3
- rs1[20:17]: registro que contiene los 32 bits a guardar
- reservado [16:0]

Aplica para: VSTR, VCLR, VAUTH, VLOGOUT

Convenciones:

- VSTR slot, palabra, rs1
- VLD rs1, slot, palabra
- VAUTH rs1
- VLOGOUT

Ejemplo: VSTR, 0, 0, r1, donde slot = 0, palabra = 0, rs1 = r1

4.6 Formato T (TEA)

Usado por instrucciones privilegiadas para acelerar operaciones asociadas al algoritmo TEA.

Campos:

- OpCode [31:27]: Código de operación (5 bits)
- rd[26:23]: Registro operando fuente (3 bits)
- rs1[22:19]: Registro operando fuente (3 bits)
- rs2[18:15]: Registro operando fuente (3 bits)
- rs3[14:11]: Registro operando fuente (3 bits)
- imm/offset [10:0]: Inmediato con extensión de signo (11-19 bits)

Aplica a: BEQADD, XORTEA

Para XORTEA, se usan tres registros fuente.

Para BEQADD, el campo rs3 puede representar el índice que se incrementa, y imm representa el destino del salto.

5. Set de Instrucciones

5.1 Instrucciones Aritméticas

Instrucción	Opcode	Formato	Parámetros	Descripción
ADD	00100	R	rd, rs1, rs2	Suma dos registros.
SUB	00101	R	rd, rs1, rs2	Resta dos registros.
MUL	01011	R	rd, rs1, rs2	Multiplica dos registros

CMP	01010	R	rs1, rs2	Compara dos registros mediante resta y actualiza banderas.
MOV	01100	I	rd, imm	Carga un valor inmediato en rd.

5.2 Instrucciones Lógicas y desplazamiento

Instrucción	Opcode	Formato	Parámetros	Descripción
AND	01101	R	rd, rs1, rs2	AND bit a bit.
OR	00110	R	rd, rs1, rs2	OR bit a bit.
XOR	00111	R	rd, rs1, rs2	XOR bit a bit.
SRL	01000	R	rs1, rs2, rs2	Desplazamiento lógico a la derecha usando registro.
SLL	01001	R	rs1, rs2, rs2	Desplazamiento lógico a la izquierda usando registro.
SRLI	10101	I	rd, rs1, imm	Desplazamiento lógico inmediato a la derecha. Privilegiada para TEA .
SLLI	10110	I	rd, rs1, imm	Desplazamiento lógico inmediato a la izquierda. Privilegiada para TEA.

5.3 Instrucciones de Memoria

Instrucción	Opcode	Formato	Parámetros	Descripción
LD	00000	M	rd, offset(rs1)	Carga un valor desde memoria a un registro.
ST	00001	M	rd, offset(rs1)	Guarda el contenido de un registro en memoria.

5.4 Instrucciones de Control de flujo

Instrucción	Opcode	Formato	Parámetros	Descripción
BEQ	00010	J	rs1, rs2, etiqueta	Salta si rs1 == rs2
JMP	00011	J	etiqueta	Salto incondicional
BEADD	10011	T	rs1, rs2, index, etiqueta	Instrucción privilegiada TEA: compara, incrementa

				índice y permite control de loop.
--	--	--	--	-----------------------------------

5.5 Instrucciones Especializadas (Seguridad)

Instrucción	Opcode	Formato	Parámetros	Descripción
VSTR	01110	K	rs1, slot, palabra	Guarda una palabra en la bóveda. Requiere autenticación.
VLD	01111	K	rs1, slot, palabra	Carga una palabra desde la bóveda a un registro seguro interno. Requiere autenticación.
VCLR	10000	K	slot	Borra una llave completa del slot indicado. Requiere autenticación.
VAUTH	10001	K	rs1	Autentica al procesador comparando rs1 contra una contraseña secreta interna.
VLOGOUT	10010	K	----	Cierra la sesión autenticada.

5.6 Instrucciones Especializadas (TEA)

Instrucción	Opcode	Formato	Parámetros	Descripción
XORTEA	10100	T	Rd, rs1, rs2, rs3	Calcula XOR de tres operandos. Requiere autenticación.
BEADD	10011	T	rs1, rs2, index, etiqueta	Controla el loop de TEA con comparación e incremento. Requiere autenticación.
ADDK	10111	T	Reg[rd] = Reg[rs1] + k_reg[imm[1:0]] (Priv)	Suma el contenido de un registro (rs1) con un valor tomado de un registro especial (k_reg) seleccionado por un índice, y guarda el resultado en rd.

SUBK	11000	T	Reg[rd] = Reg[rs1] - k_reg[imm[1:0]] (Priv)	Resta al contenido de un registro (rs1) un valor de un registro especial (k_reg) seleccionado por un índice, y guarda el resultado en rd.
------	-------	---	--	---

5.7 Instrucción nula

Instrucción	Opcode	Formato	Parámetros	Descripción
NOP	11111	R	-----	Coloca todos los valores en su valor por defecto que es cero.

Nota Importante 1 - Operaciones Privilegiadas: Las instrucciones marcadas como Privilegiadas requieren que el bit AUTH del registro de estado esté activo (AUTH = 1). Si se intenta ejecutar sin autenticación:

- La operación no se ejecuta (result = 0)
- Se activa el bit EXC (Exception) en el registro de estado

Nota Importante 2 - Operandos: aunque tenemos 32 registros direccionables con 5 bits, la codificación que se utilizó es usar campos de 3 bits y extenderlo a 5 bits, para así tener el total de 32 registros.

6. Detalles de ejecución y Comportamiento

El sistema completo integra herramientas externas y módulos de hardware para permitir el procesamiento y cifrado de archivos reales del simulador del procesador.

6.1 Flujo general del sistema

El funcionamiento del sistema se divide en tres etapas principales:

1. Carga de datos
2. Ejecución del CPU
3. Extracción de resultados

6.1.1 Carga de archivos en memoria

El proceso inicia con un archivo de entrada, el cual puede ser de distintos tipos:

- .txt, .png, .jpg, .bmp, .bin, .c, .py

Este archivo es procesador por la herramienta: load_file.py

Funcionamiento de load_file.py

- Lee el archivo como una secuencia de bytes
- Convierte cada byte a formato hexadecimal

- Genera un archivo .mem compatible con iVerilog
- Permite especificar la dirección inicial de carga en memoria

Como resultado genera el archivo program.mem. Y este archivo se carga en la RAM del procesador.

6.1.2 Ejecución del CPU

El archivo program.mem contiene las instrucciones que ejecuta el procesador.

Durante la simulación:

1. Instruction_fetch carga program.mem
2. El CPU ejecuta instrucciones secuencialmente
3. Los datos se leen desde data_mem
4. Se ejecutan operaciones de cifrado mediante instrucciones TEA

Flujo interno: program.mem ---- instruction_fetch---- decode ----control_unit --- datapath

El datapath puede:

- Leer datos desde RAM (LD)
- Escribir datos en RAM (ST)
- Acceder a la bóveda (VSTR, VLD, VCLR)
- Ejecutar operaciones criptográficas (XORTEA, BEQADD)

6.1.3 Autenticación y seguridad

Antes de ejecutar instrucciones sensibles, el procesador debe autenticarse: VAUTH r0

Si la autenticación es exitosa:

- Se habilita el acceso a bóveda
- Se permite el uso de instrucciones TEA
- Se activa el uso de instrucciones TEA
- Se activa el flag AUTH

Si no:

- Se activa EXC
- Se bloquea la operación (Access_denied)

6.1.4 Proceso de cifrado

Una vez cargados los datos y ejecutado el programa:

- EL CPU procesa los datos en bloques de 64 bits
- Aplica el algoritmo TEA mediante instrucciones especializadas
- Escribe los resultados nuevamente en la RAM

6.1.5 Extracción de resultados

Funcionamiento de extract_data.py

- Lee el archivo .mem generado después de cifrar los datos en el CPU.
- Reconstruye los bytes desde una dirección específica.

- Genera un archivo binario de salida.

Al final se obtiene un archivo, que contiene los datos cifrados.

7. Direcccionamiento

7.1 Modos de Direcccionamiento

1. Inmediato: Operando es constante (Formato I)
2. Registro: Operando en registro (Formato R)
3. Directo: Dirección en memoria (instrucción LD/ST)
4. Relativo: Offset desde PC (instrucciones de rama)

8. Seguridad y Extensiones Criptográficas

8.1 Bit de Autenticación (AUTH)

El bit AUTH en el registro de estado controla el acceso a operaciones privilegiadas:

- AUTH = 0: Solo instrucciones no privilegiadas permitidas
- AUTH = 1: Acceso a instrucciones privilegiadas (XORTEA, VREAD, VWRITE, VAUTH)

La autenticación se realiza mediante la instrucción VAUTH, que compara una contraseña almacenada en la bóveda con un valor proporcionado.

8.2 Bóveda de Llaves (Key Vault)

La bóveda de llaves es una memoria es una memoria segura interna del procesador utilizada para almacenar llaves criptográficas. Su objetivo es evitar que las llaves sean manipuladas como datos normales en la RAM o en el banco de registros generales.

El módulo implementa 4 slots, donde cada slot almacena una llave de 128 bits dividida en 4 palabras de 32 bits. La estructura interna puede representarse como vault[slot][palabra]

8.2.1 Organización de la bóveda

Slot	Palabra 0	Palabra 0	Palabra 0	Palabra 0	Tamaño total
Slot 0	vault[0][0]	vault[0][1]	vault[0][2]	vault[0][3]	128 bits
Slot 1	vault[1][0]	vault[1][1]	vault[1][2]	vault[1][3]	128 bits
Slot 2	vault[2][0]	vault[2][1]	vault[2][2]	vault[2][3]	128 bits
Slot 3	vault[3][0]	vault[3][1]	vault[3][2]	vault[3][3]	128 bits

8.2.2 Registros seguros internos de la Bóveda

La bóveda también posee registros internos seguros k_reg0, k_reg1, k_reg2 y k_reg3. Estos registros permiten cargar temporalmente palabras de una llave para ser utilizadas por instrucciones criptográficas, sin exponerlas al banco de registros general.

Registro Seguro	Contenido
-----------------	-----------

8.2.3

Instrucciones

k_reg0	Palabra 0 de una llave seleccionada
k_reg1	Palabra 1 de una llave seleccionada
k_reg2	Palabra 2 de una llave seleccionada

Asociadas

Instrucción	Señal de control	Función	Requiere Autenticación
VSTR	vault_write	Guarda una palabra de 32 bits en un slot de la bóveda.	Si
VLD	vault_load_secure	Carga una palabra de la bóveda hacia un registro seguro interno.	Si
VCLR	vault_clear	Borra las 4 palabras de un slot completo.	Si
VAUTH	auth_check	Autentica al procesador comparando el valor de rs1 con una contraseña interna.	No
VLOGOUT	vault_clear	Cierra la sesión autenticada.	Si

Si una instrucción intenta acceder a la bóveda sin que auth_status =1, el módulo activa exc_out, indicando una excepción por acceso no autorizado.

8.3 Mecanismo de Autenticación

El procesador también incorpora un módulo de autenticación encargado de controlar el acceso a operaciones privilegiadas, como el uso de la bóveda de llaves y las instrucciones criptográficas. Este mecanismo garantiza que únicamente código autorizado pueda manipular información sensible.

8.3.1 Funcionamiento del módulo

El módulo mantiene un estado interno de autenticación mediante señal: auth_status

- Cuando auth_status = 1, el procesador puede ejecutar instrucciones privilegiadas.
- Cuando auth_status = 0, dichas instrucciones son bloqueadas.

8.3.2 Parámetros de Seguridad

Parámetro	Valor	Descripción
SECRET	0XA5A5A5A5	Contraseña interna del procesador
AUTH_TIMEOUT	64 ciclos	Duración de la sesión autenticada

Se

implementa entonces un mecanismo de autenticación temporal basado en contraseña, con soporte para expiración automática y control de acceso a instrucciones privilegiadas, reforzando la seguridad del procesador.

9. Conclusiones

El proyecto permitió diseñar una ISA tipo RISC orientada a aplicaciones de seguridad informática, integrando instrucciones tradicionales de procesamiento con extensiones específicas para autenticación, manejo seguro de llaves y apoyo al cifrado TEA.

La separación entre memoria de instrucciones y memoria de datos mediante una arquitectura Harvard simplificada facilitó la organización del procesador, redujo conflictos de acceso y permitió distinguir claramente entre el programa ejecutado y los datos procesados.

La implementación de la bóveda de llaves y del módulo de autenticación fortaleció el diseño de seguridad, ya que las llaves no se exponen directamente al banco de registros general ni a la RAM. Además, el uso de VAUTH, VLOGOUT y un temporizador de expiración permite controlar temporalmente el acceso a instrucciones privilegiadas.

La microarquitectura modular en SystemVerilog permitió dividir el procesador en componentes independientes como fetch, decode, control unit, datapath, ALU, memoria, bóveda y autenticación, facilitando la validación mediante testbenches individuales y pruebas de integración.

Finalmente, la herramienta de carga y extracción de archivos permitió conectar la simulación con datos reales, demostrando que la arquitectura puede operar sobre archivos de distintos formatos y aplicar procesamiento criptográfico dentro de la memoria simulada.